

零声教育 Darren

课程: [埃梯零声 \(0voice.com\)](https://0voice.com)

特别注意:

课程使用的ROS2版本必须在 Ubuntu24.04 desktop 桌面版运行

Ubuntu24.04桌面版下载地址: <https://mirrors.aliyun.com/ubuntu-releases/24.04/ubuntu-24.04.2-desktop-amd64.iso>

课件地址: https://gitlab.0voice.com/2503_vip/ros2-doc

源码地址: https://gitlab.0voice.com/2503_vip/ros2-src

1 ros2简介

ROS 2 (Robot Operating System 2) 是机器人操作系统的最新版本, 是一个开源的机器人中间件框架, 为机器人软件开发提供了一系列工具、库和约定。

ROS 2 的主要特点

1. **实时性支持**: 相比 ROS 1, ROS 2 提供了更好的实时性能
2. **多平台支持**: 支持 Linux、Windows 和 macOS
3. **多种通信方式**: 支持 DDS (Data Distribution Service) 作为底层通信机制
4. **改进的安全特性**: 支持加密通信和访问控制
5. **多语言支持**: C++、Python 等

ROS 2 的核心概念

- **节点 (Nodes)**: 执行计算的进程
- **话题 (Topics)**: 发布/订阅模式的通信通道
- **服务 (Services)**: 请求/响应模式的通信
- **动作 (Actions)**: 长时间运行的任务接口
- **参数 (Parameters)**: 可配置的节点设置

ROS 2 与 ROS 1 的主要区别

1. **架构**: ROS 2 使用 DDS 代替 ROS 1 的自定义通信层
2. **实时性**: ROS 2 设计时考虑了实时系统需求
3. **跨平台**: ROS 2 原生支持 Windows
4. **安全性**: ROS 2 内置安全功能

ROS 2 的应用领域

- 自动驾驶
- 工业机器人
- 服务机器人
- 无人机系统
- 研究原型开发

2 ros2环境搭建

Ubuntu24.04桌面版本 + ros2 jazzy

检查操作系统版本

确保你的系统版本是Ubuntu 24.04，使用以下命令来检查当前的系统版本。

```
lsb_release -a
```

正确时提示：

```
Description:    Ubuntu 24.04.2 LTS
Release:        24.04
```

设置编码格式

ROS 2需要UTF-8编码支持

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

export LANG=en_US.UTF-8
```

添加软件源

Ubuntu 24.04通常会默认启用Universe存储库

```
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update
```

安装必要的工具

```
sudo apt install curl -y
```

设置ROS 2软件源:

- 下载ROS 2的GPG密钥并添加到系统中:

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

- 将ROS 2的软件源添加到系统的源列表中(建议使用清华源):

```
# 优先使用清华的源, 后续安装ros2时更快, ros2需要3GB的下载
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] https://mirrors.tuna.tsinghua.edu.cn/ros2/ubuntu noble main" |
sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 官方源(用了清华源这里就不需要了), 官方源下载很慢。
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

安装ros2

安装ROS2建议首先更新并升级你的apt代码仓库, 在终端中输入以下命令:

```
sudo apt update
sudo apt upgrade
```

然后, 你可以开始安装Desktop版, 里面包含了ROS、RViz、demos和教程。

```
sudo apt install ros-jazzy-desktop
```

```
linux-common linux-dev va-driver-all vdpau-driver-all vlc5 x11
0 upgraded, 986 newly installed, 0 to remove and 1 not upgraded.
Need to get 653 MB of archives.
After this operation, 2,930 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

设置环境变量

- 每次打开终端时, 都需要设置ROS 2的环境变量。可以通过以下命令设置:

```
source /opt/ros/jazzy/setup.bash
```

- 为了避免每次都要手动设置, 可以将此命令添加到 ~/.bashrc 文件中:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

跑例子验证

首先，打开一个终端，输入以下命令配置环境，并打开一个C++ talker:

```
ros2 run demo_nodes_cpp talker
```

你会看到talker一直在发布消息并输出日志“Publishing: ...”

```
lqf@lqf-ros2:~/ros2_jazzy$ ros2 run demo_nodes_cpp talker
[INFO] [1749055871.342984622] [talker]: Publishing: 'Hello World: 1'
[INFO] [1749055872.342843171] [talker]: Publishing: 'Hello World: 2'
[INFO] [1749055873.342812124] [talker]: Publishing: 'Hello World: 3'
[INFO] [1749055874.342757586] [talker]: Publishing: 'Hello World: 4'
[INFO] [1749055875.342827931] [talker]: Publishing: 'Hello World: 5'
[INFO] [1749055876.342663871] [talker]: Publishing: 'Hello World: 6'
[INFO] [1749055877.342663085] [talker]: Publishing: 'Hello World: 7'
[INFO] [1749055878.342782829] [talker]: Publishing: 'Hello World: 8'
[INFO] [1749055879.342674881] [talker]: Publishing: 'Hello World: 9'
```

然后再打开另一个终端，同样也是配置环境，然后启动一个Python listener

```
# 重启系统后 .bashrc 才每个终端生效，所以这里另起终端时还是要source下
source ~/.bashrc
```

```
ros2 run demo_nodes_py listener
```

这时，可以看到listener接收到了消息并持续输出日志“I heard: ...”

```
lqf@lqf-ros2:~$ ros2 run demo_nodes_py listener
[INFO] [1749055878.364780005] [listener]: I heard: [Hello World: 8]
[INFO] [1749055879.344437091] [listener]: I heard: [Hello World: 9]
[INFO] [1749055880.343607806] [listener]: I heard: [Hello World: 10]
[INFO] [1749055881.343889794] [listener]: I heard: [Hello World: 11]
[INFO] [1749055882.344135373] [listener]: I heard: [Hello World: 12]
[INFO] [1749055883.343937079] [listener]: I heard: [Hello World: 13]
[INFO] [1749055884.345010726] [listener]: I heard: [Hello World: 14]
[INFO] [1749055885.344544264] [listener]: I heard: [Hello World: 15]
```

如果你成功看到以上画面，恭喜你，ROS2安装成功了！

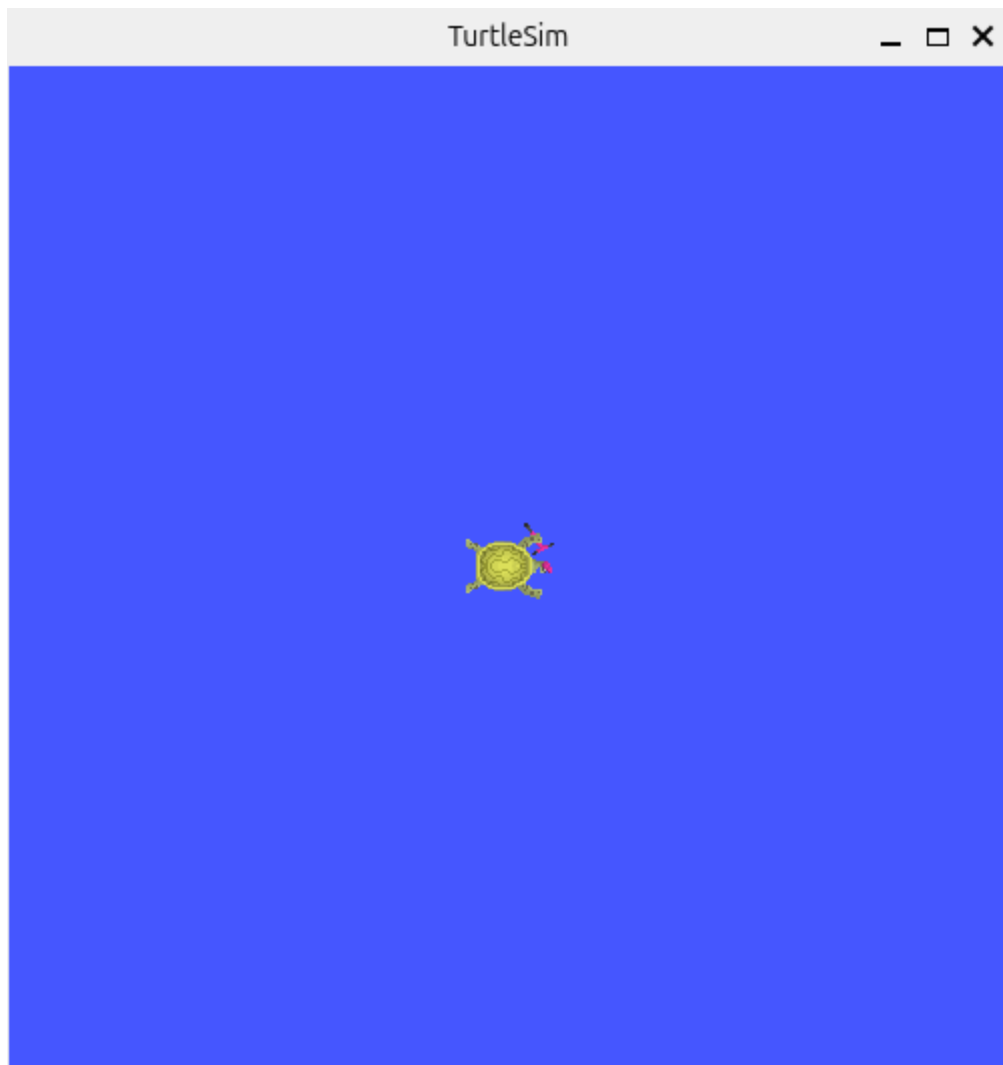
海龟仿真实践

需要进到桌面启动终端

首先启动一个终端

```
ros2 run turtlesim turtlesim_node
```

会出现一个小乌龟



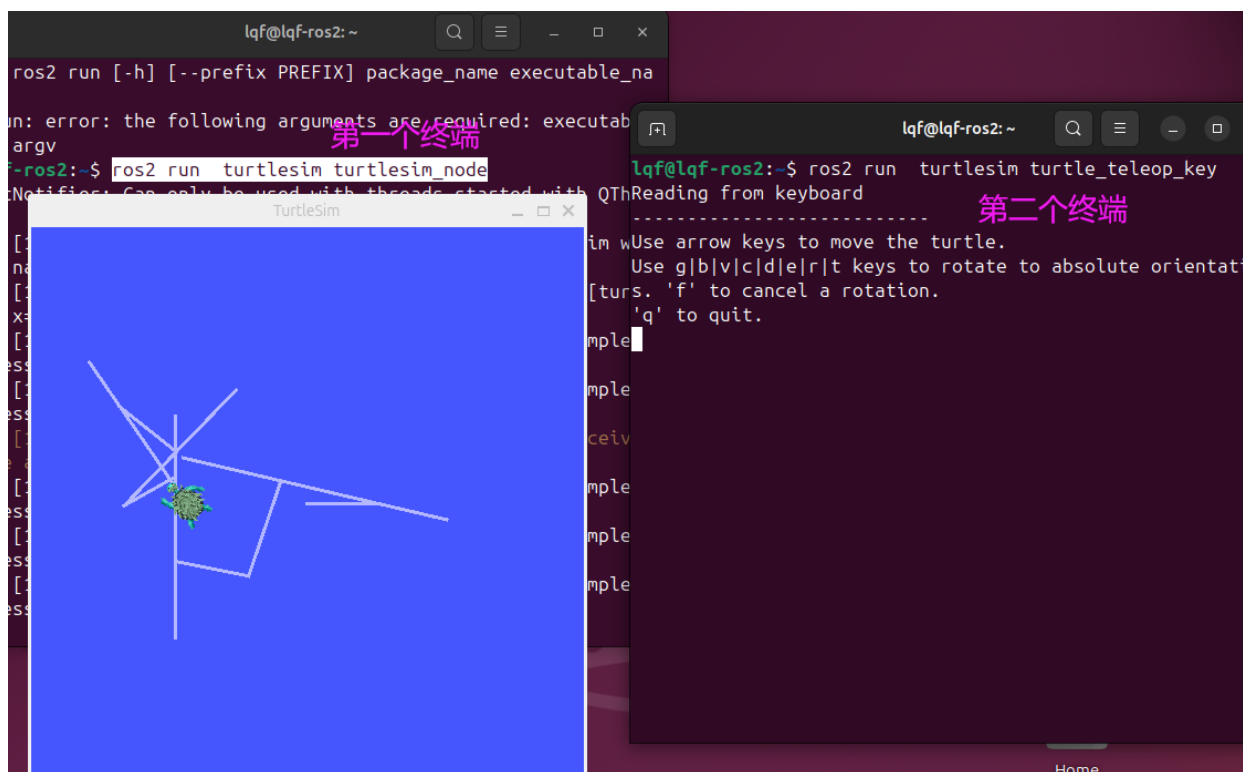
然后再启动一个终端

```
ros2 run turtlesim turtle_teleop_key
```

```
lqf@lqf-ros2: ~  
lqf@lqf-ros2:~$ ros2 run turtlesim turtle_teleop_key  
Reading from keyboard  
-----  
Use arrow keys to move the turtle.  
Use g|b|v|c|d|e|r|t keys to rotate to absolute orientations. 'f'  
' ' to cancel a rotation.  
'q' to quit.  
█
```

鼠标点在上面

就可以按照提示的信息，使用箭头键来控制海龟行走了。



3 ros2 机器人开发基础

3.1 ROS2开发流程

ROS2 机器人开发流程为：

- 1.创建工作空间;
- 2.创建功能包;
- 3.编写源代码;
- 4.设置编译规则;
- 5.编译和调试;
- 6.功能运行。

1. 创建工作空间 (Workspace Creation)

工作空间是ROS2项目的根目录，用于组织和管理多个功能包。

- **目的：**为ROS2项目提供统一的开发环境
- **结构：**通常包含 `src/`、`build/`、`install/`、`log/` 等目录
- **命令示例：**

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

2. 创建功能包 (Package Creation)

功能包是ROS2的基本代码组织单元，包含节点、消息、服务等。

- **目的：**模块化开发，便于代码复用和维护
- **包含内容：**源代码、配置文件、依赖关系等
- **命令示例：**

```
cd ~/ros2_ws/src  
ros2 pkg create --build-type ament_python my_package
```

3. 编写源代码 (Source Code Development)

实现具体的机器人功能逻辑。

- **主要内容：**
 - 节点 (Node)：ROS2的基本执行单元
 - 话题 (Topic)：异步通信机制
 - 服务 (Service)：同步通信机制
 - 动作 (Action)：长时间运行的任务
- **编程语言：**主要支持Python和C++

4. 设置编译规则 (Build Configuration)

配置项目的编译和依赖关系。

- **主要文件：**

- `package.xml`：包的元信息和依赖声明
- `setup.py` (Python) 或 `CMakeLists.txt` (C++)：编译配置
- **功能**：定义依赖项、编译选项、安装规则等

5. 编译和调试 (Build and Debug)

将源代码编译成可执行文件并进行测试。

- **编译命令**：

```
cd ~/ros2_ws  
colcon build
```

- **调试工具**：
 - 日志系统
 - RQT工具链
 - 单元测试框架

6. 功能运行 (Execution)

启动和运行已开发的ROS2节点和系统。

- **运行步骤**：
 1. 设置环境变量：`source ~/ros2_ws/install/setup.bash`
 2. 启动节点：`ros2 run package_name node_name`
 3. 使用launch文件启动多个节点
- **监控工具**：RQT、rviz2等可视化工具

这个开发流程是循环迭代的，在实际开发中经常需要在不同阶段之间切换，持续改进和优化机器人系统的功能。

3.2 工作空间

3.2.1 工作空间是什么

定义和概念

- **工作空间**：是一个包含ROS2功能包的目录结构，用于组织、构建和管理ROS2项目
- **作用**：提供一个独立的开发环境，隔离不同项目的依赖和配置

目录结构

典型的工作空间结构

```
ros2_ws/                                # 工作空间根目录
├─ src/                                  # 源代码目录
│   ├─ package1/                        # 功能包1（Python包示例）
│   │   ├─ package1/                    # Python模块目录
│   │   │   ├─ __init__.py              # Python包初始化文件
│   │   │   ├─ node1.py                 # 节点源代码
│   │   │   └─ module1.py               # 自定义模块
│   │   └─ resource/                    # 资源文件目录
│   │       └─ package1                 # 包标识文件
│   └─ test/                            # 测试文件目录
│       ├─ test_copyright.py            # 版权测试
│       ├─ test_flake8.py               # 代码风格测试
│       └─ test_pep257.py               # 文档字符串测试
│   └─ launch/                          # 启动文件目录
│       └─ package1_launch.py           # 启动文件
│   └─ config/                          # 配置文件目录
│       └─ params.yaml                  # 参数配置文件
│   └─ package.xml                      # 包元信息文件
│   └─ setup.py                        # Python包安装配置
│   └─ setup.cfg                        # 安装配置
│
│   └─ package2/                        # 功能包2（C++包示例）
│       ├─ src/                         # C++源代码目录
│       │   ├─ node2.cpp                # 节点源代码
│       │   └─ lib2.cpp                 # 库源代码
│       └─ include/                     # 头文件目录
│           └─ package2/                # 包头文件目录
│               ├─ node2.hpp             # 节点头文件
│               └─ lib2.hpp              # 库头文件
│       └─ launch/                      # 启动文件目录
│           └─ package2_launch.py         # 启动文件
│       └─ config/                      # 配置文件目录
│           └─ config.yaml               # 配置文件
│       └─ msg/                         # 自定义消息定义
│           └─ CustomMsg.msg             # 消息定义文件
│       └─ srv/                         # 自定义服务定义
│           └─ CustomSrv.srv             # 服务定义文件
│       └─ action/                      # 自定义动作定义
│           └─ CustomAction.action        # 动作定义文件
│       └─ test/                        # 测试文件目录
│           ├─ test_node2.cpp            # 单元测试
│           └─ test_integration.cpp       # 集成测试
│       └─ package.xml                  # 包元信息文件
│       └─ CMakeLists.txt               # CMake编译配置
│
│   └─ interfaces_package/              # 接口包示例
│       └─ msg/                         # 消息定义目录
│           └─ Status.msg                # 状态消息
```

```

|       |   └─ SensorData.msg      # 传感器数据消息
|       └─ srv/                    # 服务定义目录
|       |   └─ GetData.srv         # 获取数据服务
|       └─ action/                 # 动作定义目录
|       |   └─ Navigate.action     # 导航动作
|       └─ package.xml             # 包元信息
|       └─ CMakeLists.txt          # CMake配置
|
└─ build/                          # 编译中间文件目录
|
└─ install/                        # 安装文件目录
|
└─ log/                            # 编译日志目录

```

工作空间类型

- **Overlay工作空间**：叠加在已有工作空间之上
- **Underlay工作空间**：底层工作空间（如ROS2安装目录）

3.2.2 创建工作空间的方式

方法一：手动创建

```

# 创建工作空间目录
mkdir -p ~/ros2_ws/src

# 进入工作空间
cd ~/ros2_ws

# 初始化src目录（可选）
cd src

```

方法二：克隆现有项目

```

# 创建工作空间
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src

# 克隆ROS2示例
git clone https://github.com/ros2/examples.git

```

验证工作空间结构

```

# 检查目录结构
tree ~/ros2_ws
# 或者
ls -la ~/ros2_ws

```

3.2.3 设置编译环境

工作空间环境设置

```
# 编译后设置工作空间环境
cd ~/ros2_ws
source install/setup.bash

# 验证包是否可用
ros2 pkg list | grep your_package_name
```

永久环境配置

```
# 配置在系统的环境变量中
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc

# 重新加载bashrc
source ~/.bashrc
```

环境变量检查

```
# 检查重要的ROS2环境变量
echo "ROS_DISTRO: $ROS_DISTRO"
echo "ROS_VERSION: $ROS_VERSION"
echo "ROS_DOMAIN_ID: $ROS_DOMAIN_ID"
echo "AMENT_PREFIX_PATH: $AMENT_PREFIX_PATH"

# 检查Python路径
echo "PYTHONPATH: $PYTHONPATH"
```

3.3 功能包 机器人功能分类

3.3.1 什么是功能包

定义和概念

- **功能包**：ROS2中代码组织的基本单元，包含实现特定功能的节点、库、配置文件等
- **模块化设计**：每个功能包专注于解决特定问题，便于代码复用和维护
- **依赖管理**：通过功能包管理代码间的依赖关系

功能包的作用

功能包的核心作用：

- └─ 代码组织 # 将相关功能代码组织在一起
- └─ 依赖管理 # 声明和管理外部依赖
- └─ 接口定义 # 定义消息、服务、动作接口
- └─ 资源管理 # 管理配置文件、启动文件等资源
- └─ 版本控制 # 独立的版本管理和发布

功能包类型

- **可执行包**：包含节点的功能包
- **库包**：提供可重用库的功能包
- **接口包**：定义消息、服务、动作的功能包
- **元包 (Metapackage)**：组织相关功能包的集合

3.3.2 创建功能包

Python功能包创建

我们课程里不讲python包，这里只需了解。

```
# 进入工作空间src目录
cd ~/ros2_ws/src

# 创建Python功能包
ros2 pkg create --build-type ament_python my_python_pkg
# 创建后在当前目录 增加了 my_python_pkg 目录

# 带依赖的Python包创建
ros2 pkg create --build-type ament_python my_python_pkg \
  --dependencies rclpy std_msgs sensor_msgs geometry_msgs

# 创建带节点的Python包
ros2 pkg create --build-type ament_python my_python_pkg \
  --dependencies rclpy std_msgs \
  --node-name my_node
```

C++功能包创建

课程里主要讲C++的使用方式。

```

# 创建C++功能包
ros2 pkg create --build-type ament_cmake my_cpp_pkg
# 创建后在当前目录 增加了 my_cpp_pkg 目录

# 带依赖的C++包创建
ros2 pkg create --build-type ament_cmake my_cpp_pkg \
  --dependencies rclcpp std_msgs sensor_msgs geometry_msgs

# 创建带节点的C++包
ros2 pkg create --build-type ament_cmake my_cpp_pkg \
  --dependencies rclcpp std_msgs \
  --node-name my_node

```

3.3.3 功能包的结构 (Package Structure)

Python功能包结构

我们课程里不讲python包，这里只需了解。

my_python_pkg/	# 功能包根目录
├─ my_python_pkg/	# Python模块目录
└─ __init__.py	# Python包初始化文件
└─ my_node.py	# 节点源文件
└─ my_module.py	# 自定义模块
└─ submodule/	# 子模块目录
└─ __init__.py	# 子模块初始化
└─ helper.py	# 辅助功能模块
├─ launch/	# 启动文件目录
└─ my_launch.py	# Python启动文件
└─ my_launch.xml	# XML启动文件
├─ config/	# 配置文件目录
└─ params.yaml	# 参数配置文件
└─ robot_config.yaml	# 机器人配置文件
├─ resource/	# 资源文件目录
└─ my_python_pkg	# 包标识文件
├─ test/	# 测试文件目录
└─ test_copyright.py	# 版权测试
└─ test_flake8.py	# 代码风格测试
└─ test_pep257.py	# 文档字符串测试
└─ test_my_node.py	# 自定义单元测试
├─ package.xml	# 包元信息文件
├─ setup.py	# Python包安装配置
├─ setup.cfg	# 安装配置
└─ README.md	# 包说明文档

Python功能包关键文件详解

setup.py文件:

```
from setuptools import setup

package_name = 'my_python_pkg'

setup(
    name=package_name,
    version='0.0.1',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        # 添加启动文件
        ('share/' + package_name + '/launch', ['launch/my_launch.py']),
        # 添加配置文件
        ('share/' + package_name + '/config', ['config/params.yaml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Your Name',
    maintainer_email='326873713@qq.com',
    description='Python package description',
    license='Apache License 2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'my_node = my_python_pkg.my_node:main',
        ],
    },
)
```

节点源文件示例:

```
#!/usr/bin/env python3
# my_python_pkg/my_node.py

import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from geometry_msgs.msg import Twist

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')

        # 创建发布者
        self.publisher = self.create_publisher(String, 'topic', 10)
```

```

# 创建订阅者
self.subscription = self.create_subscription(
    Twist, 'cmd_vel', self.listener_callback, 10)

# 创建定时器
timer_period = 0.5
self.timer = self.create_timer(timer_period, self.timer_callback)

self.get_logger().info('Node initialized')

def timer_callback(self):
    msg = String()
    msg.data = 'Hello world'
    self.publisher.publish(msg)

def listener_callback(self, msg):
    self.get_logger().info(f'Received: {msg}')

def main(args=None):
    rclpy.init(args=args)
    node = MyNode()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

C++功能包结构

my_cpp_pkg/	# 功能包根目录
├─ src/	# 源代码目录
└─ my_node.cpp	# 节点源文件
└─ my_library.cpp	# 库源文件
└─ utils/	# 工具类源文件
└─ helper.cpp	# 辅助功能实现
├─ include/	# 头文件目录
└─ my_cpp_pkg/	# 包头文件目录
└─ my_node.hpp	# 节点头文件
└─ my_library.hpp	# 库头文件
└─ utils/	# 工具类头文件
└─ helper.hpp	# 辅助功能头文件
├─ launch/	# 启动文件目录
└─ my_launch.py	# Python启动文件

	└─ my_launch.xml	# XML启动文件
	└─ config/	# 配置文件目录
	└─ params.yaml	# 参数配置文件
	└─ robot_config.yaml	# 机器人配置文件
	└─ test/	# 测试文件目录
	└─ test_my_node.cpp	# 节点单元测试
	└─ test_my_library.cpp	# 库单元测试
	└─ package.xml	# 包元信息文件
	└─ CMakeLists.txt	# CMake编译配置
	└─ README.md	# 包说明文档

C++功能包关键文件详解

CMakeLists.txt文件:

```
cmake_minimum_required(VERSION 3.8)
project(my_cpp_pkg)

# 编译选项
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)

# 包含头文件目录
include_directories(include)

# 创建库
add_library(${PROJECT_NAME}_lib
    src/my_library.cpp
    src/utils/helper.cpp
)

# 库的依赖
ament_target_dependencies(${PROJECT_NAME}_lib
    rclcpp
    std_msgs
    geometry_msgs
)

# 创建可执行文件
add_executable(my_node src/my_node.cpp)

# 可执行文件的依赖
ament_target_dependencies(my_node
    rclcpp
    std_msgs
```

```

    geometry_msgs
)

# 链接库
target_link_libraries(my_node ${PROJECT_NAME}_lib)

# 安装可执行文件
install(TARGETS my_node
  DESTINATION lib/${PROJECT_NAME}
)

# 安装库
install(TARGETS ${PROJECT_NAME}_lib
  DESTINATION lib
)

# 安装头文件
install(DIRECTORY include/
  DESTINATION include
)

# 安装启动文件
install(DIRECTORY launch/
  DESTINATION share/${PROJECT_NAME}/launch
)

# 安装配置文件
install(DIRECTORY config/
  DESTINATION share/${PROJECT_NAME}/config
)

# 导出依赖
ament_export_include_directories(include)
ament_export_libraries(${PROJECT_NAME}_lib)
ament_export_dependencies(rclcpp std_msgs geometry_msgs)

# 测试
if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  ament_lint_auto_find_test_dependencies()

  # 添加单元测试
  find_package(ament_cmake_gtest REQUIRED)
  ament_add_gtest(test_my_node test/test_my_node.cpp)
  target_link_libraries(test_my_node ${PROJECT_NAME}_lib)
endif()

ament_package()

```

C++节点源文件示例:

```
// src/my_node.cpp
#include "my_cpp_pkg/my_node.hpp"

MyNode::MyNode() : Node("my_node")
{
    // 创建发布者
    publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);

    // 创建订阅者
    subscription_ = this->create_subscription<geometry_msgs::msg::Twist>(
        "cmd_vel", 10,
        std::bind(&MyNode::listener_callback, this, std::placeholders::_1));

    // 创建定时器
    timer_ = this->create_wall_timer(
        std::chrono::milliseconds(500),
        std::bind(&MyNode::timer_callback, this));

    RCLCPP_INFO(this->get_logger(), "Node initialized");
}

void MyNode::timer_callback()
{
    auto message = std_msgs::msg::String();
    message.data = "Hello world";
    publisher_>publish(message);
}

void MyNode::listener_callback(const geometry_msgs::msg::Twist::SharedPtr msg)
{
    RCLCPP_INFO(this->get_logger(), "Received twist message");
}

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MyNode>());
    rclcpp::shutdown();
    return 0;
}
```

package.xml文件 (通用)

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_cpp_pkg</name>
  <version>0.0.1</version>
```

```

<description>C++ package description</description>
<maintainer email="326873713@qq.com">Your Name</maintainer>
<license>Apache License 2.0</license>

<!-- 构建依赖 -->
<buildtool_depend>ament_cmake</buildtool_depend>

<!-- 编译依赖 -->
<depend>rc1cpp</depend>
<depend>std_msgs</depend>
<depend>geometry_msgs</depend>

<!-- 测试依赖 -->
<test_depend>ament_lint_auto</test_depend>
<test_depend>ament_lint_common</test_depend>
<test_depend>ament_cmake_gtest</test_depend>

<export>
  <build_type>ament_cmake</build_type>
</export>
</package>

```

3.3.4 编译功能包 (Package Building)

编译指定功能包

```

# 编译单个包
colcon build --packages-select my_python_pkg

# 编译多个指定包
colcon build --packages-select my_python_pkg my_cpp_pkg

# 编译包及其依赖
colcon build --packages-up-to my_python_pkg

# 编译修改过的包
colcon build --packages-changed

```

编译所有功能包

```

# 基础编译命令
cd ~/ros2_ws
colcon build

# 详细输出编译过程
colcon build --event-handlers console_direct+

# 并行编译（使用多核）

```

```
colcon build --parallel-workers 4

# 符号链接安装（开发时推荐）
colcon build --symlink-install

# 编译时显示编译器输出
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Debug

# 编译时显示详细信息
colcon build --event-handlers console_cohesion+ desktop_notification-
```

清除编译历史

```
# 删除build、install、log目录
rm -rf build/ install/ log/

# 删除指定包的编译文件
rm -rf build/my_python_pkg/ install/my_python_pkg/

# 使用colcon清理缓存
colcon build --cmake-clean-cache

# 强制重新编译所有包
colcon build --cmake-clean-first

# 清理Python字节码文件
find . -name "*.pyc" -delete
find . -name "__pycache__" -type d -exec rm -rf {} +
```

编译验证和测试

```
# 编译并运行测试
colcon build --packages-select my_cpp_pkg
colcon test --packages-select my_cpp_pkg

# 查看测试结果
colcon test-result --all

# 详细测试结果
colcon test-result --verbose

# 编译时运行代码检查
colcon build --packages-select my_python_pkg --event-handlers console_direct+
```